Hybrid QNN Training on BCCC-CIC-IDS-2017 (Resumable)

This notebook trains a Hybrid Quantum Neural Network (QNN) on the processed BCCC-CIC-IDS-2017 dataset. It includes functionality to save checkpoints after every epoch and resume training if interrupted.

In [1]:
```
import pandas as pd
import numpy as np
import os
import torch
import torch.nn as nn
import pennylane as qml
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from torch.utils.data import DataLoader, TensorDataset
from tqdm import tqdm
import warnings
warnings.filterwarnings('ignore')
```

1. Configuration & Data Loading

In [2]:
```
# Configuration
DATASET_PATH = 'Datasets/processed_data.csv'
BATCH_SIZE = 32
EPOCHS = 15
LEARNING_RATE = 0.01
N_QUBITS = 4
N_LAYERS = 2
SEED = 42

# Checkpoint Configuration
CHECKPOINT_DIR = 'checkpoints'
os.makedirs(CHECKPOINT_DIR, exist_ok=True)
CHECKPOINT_PATH = os.path.join(CHECKPOINT_DIR, 'checkpoint_ids2017.pth')

np.random.seed(SEED)
torch.manual_seed(SEED)
```

Out[2]:
```
<torch._C.Generator at 0x1c2ce4541d0>
```

In [3]:
```
# Load Data
print("Loading dataset...")
if not os.path.exists(DATASET_PATH):
    raise FileNotFoundError(f"Dataset not found at {DATASET_PATH}. Please run preprocess_da

df = pd.read_csv(DATASET_PATH)
print(f"Total samples: {len(df)}")

# Check for label column
if 'label' not in df.columns:
    raise ValueError("Column 'label' not found in dataset.")

# Separate Features and Target
X = df.drop(columns=['label']).values
y = df['label'].values

# Determine number of classes
```

```
num_classes = len(np.unique(y))
print(f"Number of classes: {num_classes}")

# Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=SEED)

# Convert to PyTorch Tensors
X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train, dtype=torch.long) # Long for CrossEntropy, Float for
X_test_tensor = torch.tensor(X_test, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test, dtype=torch.long)

# Adjust for Binary vs Multiclass
if num_classes == 2:
    y_train_tensor = y_train_tensor.float().unsqueeze(1)
    y_test_tensor = y_test_tensor.float().unsqueeze(1)
    criterion = nn.BCEWithLogitsLoss()
    output_dim = 1
    print("Binary Classification detected.")
else:
    criterion = nn.CrossEntropyLoss()
    output_dim = num_classes
    print("Multiclass Classification detected.")

# Create DataLoaders
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
test_dataset = TensorDataset(X_test_tensor, y_test_tensor)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)

print(f"Training samples: {len(X_train)}")
print(f"Testing samples: {len(X_test)}")
print(f"Input Feature Dimension: {X_train.shape[1]}")
```

```
Loading dataset...
Total samples: 840549
Number of classes: 14
Multiclass Classification detected.
Training samples: 672439
Testing samples: 168110
Input Feature Dimension: 116
```

2. Hybrid QNN Architecture

In [4]:

```
# Quantum Device
dev = qml.device("default.qubit", wires=N_QUBITS)

@qml.qnode(dev, interface="torch")
def qnn_layer(inputs, weights):
    # Encode classical data (Angle Embedding)
    for i in range(N_QUBITS):
        qml.RX(inputs[i], wires=i)

    # Variational layers
    qml.templates.StronglyEntanglingLayers(weights, wires=range(N_QUBITS))

    # Measurement
    return [qml.expval(qml.PauliZ(i)) for i in range(N_QUBITS)]

class HybridQNN(nn.Module):
    def __init__(self, input_dim, n_qubits, n_layers, output_dim):
        super().__init__()
        self.n_qubits = n_qubits

        # Classical Projection Layer: Reduces input_dim -> n_qubits
```

```
        self.projection = nn.Linear(input_dim, n_qubits)

        # Quantum Weights
        self.q_weights = nn.Parameter(
            0.01 * torch.randn(n_layers, n_qubits, 3)
        )

        # Classical Output Head
        self.fc = nn.Linear(n_qubits, output_dim)

    def forward(self, x):
        # 1. Project High-Dim Features -> Qubits
        x_proj = torch.tanh(self.projection(x)) * np.pi # Scale to [-pi, pi]

        # 2. Quantum Layer
        q_out = []
        for sample in x_proj:
            q = qnn_layer(sample, self.q_weights)
            q = torch.stack(q)
            q_out.append(q)

        q_out = torch.stack(q_out).float()

        # 3. Final Classification
        return self.fc(q_out)
```

In [5]:

```
# Initialize Model
input_dim = X_train.shape[1]
model = HybridQNN(input_dim, N_QUBITS, N_LAYERS, output_dim)
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)

print(model)
```

```
HybridQNN(
  (projection): Linear(in_features=116, out_features=4, bias=True)
  (fc): Linear(in_features=4, out_features=14, bias=True)
)
```

3. Training Loop with Resume Capability

In [ ]:

```
def evaluate(model, loader, is_binary):
    model.eval()
    all_preds = []
    all_labels = []
    with torch.no_grad():
        for X_batch, y_batch in loader:
            logits = model(X_batch)
            if is_binary:
                preds = torch.sigmoid(logits) > 0.5
            else:
                preds = torch.argmax(logits, dim=1)

            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(y_batch.cpu().numpy())

    # Metrics
    average_type = 'binary' if is_binary else 'weighted'
    acc = accuracy_score(all_labels, all_preds)
    prec = precision_score(all_labels, all_preds, average=average_type, zero_division=0)
    rec = recall_score(all_labels, all_preds, average=average_type, zero_division=0)
    f1 = f1_score(all_labels, all_preds, average=average_type, zero_division=0)
    return acc, prec, rec, f1

# Check for checkpoint
start_epoch = 0
```

```python
if os.path.exists(CHECKPOINT_PATH):
    print(f"Loading checkpoint from {CHECKPOINT_PATH}...")
    checkpoint = torch.load(CHECKPOINT_PATH)
    model.load_state_dict(checkpoint['model_state_dict'])
    optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
    start_epoch = checkpoint['epoch'] + 1
    print(f"Resuming training from epoch {start_epoch+1}")
else:
    print("No checkpoint found. Starting training from scratch.")

print("Starting training...")
is_binary = (num_classes == 2)

if start_epoch < EPOCHS:
    for ep in range(start_epoch, EPOCHS):
        model.train()
        total_loss = 0
        for X_batch, y_batch in tqdm(train_loader, desc=f"Epoch {ep+1}/{EPOCHS}"):
            optimizer.zero_grad()
            logits = model(X_batch)
            loss = criterion(logits, y_batch)
            loss.backward()
            optimizer.step()
            total_loss += loss.item()

        avg_loss = total_loss / len(train_loader)
        acc, prec, rec, f1 = evaluate(model, test_loader, is_binary)

        print(f"Epoch {ep+1}: Loss = {avg_loss:.4f}")
        print(f"  Test Accuracy: {acc:.4f}, Precision: {prec:.4f}, Recall: {rec:.4f}, F1: {

        # Save Checkpoint
        checkpoint = {
            'epoch': ep,
            'model_state_dict': model.state_dict(),
            'optimizer_state_dict': optimizer.state_dict(),
            'loss': avg_loss,
        }
        torch.save(checkpoint, CHECKPOINT_PATH)
        print(f"Checkpoint saved for Epoch {ep+1}")
else:
    print("Training already completed!")
```

```
Loading checkpoint from checkpoints\checkpoint_ids2017.pth...
Resuming training from epoch 11
Starting training...
Epoch 11/15: 100%|██████████| 21014/21014 [1:58:11<00:00,  2.96it/s]
```

In [9]:

```python
# Save the final trained model (keeping legacy format as well)
save_dir = 'trained_model'
os.makedirs(save_dir, exist_ok=True)
model_path = os.path.join(save_dir, 'hybrid_qnn_ids2017.pth')
torch.save(model.state_dict(), model_path)
print(f"Final Model saved to {model_path}")
```

```
Final Model saved to trained_model\hybrid_qnn_ids2017.pth
```